

Modeling Multiprogramming

Definition

Modeling multiprogramming means estimating how much CPU utilization improves when several processes are kept in memory at the same time.

1. Basic Idea

Main Concept

Multiprogramming improves CPU utilization because while one process is waiting for I/O, another process may be able to use the CPU.

Optimistic Intuition

If each process computes only a small fraction of the time and spends most of its time waiting for I/O, then keeping several processes in memory can keep the CPU busy much more often.

Why the Naive View Fails

The simple idea that “five processes each using the CPU 20% of the time will keep the CPU busy 100% of the time” is too optimistic. It assumes that all processes will never wait for I/O at the same time.

2. Probabilistic Model

Assumption

Suppose a process spends a fraction p of its time waiting for I/O. If there are n processes in memory, then the probability that **all** of them are waiting for I/O at the same time is:

$$p^n$$

CPU Utilization Formula

If all n processes are waiting, the CPU is idle. So the CPU utilization is:

$$\text{CPU utilization} = 1 - p^n$$

Meaning of Symbols

- p : fraction of time one process waits for I/O
- n : number of processes in memory
- p^n : probability that all processes are waiting
- $1 - p^n$: probability that at least one process can use the CPU

3. Effect of Degree of Multiprogramming

Degree of Multiprogramming

The number of processes kept in memory at once is called the **degree of multiprogramming**.

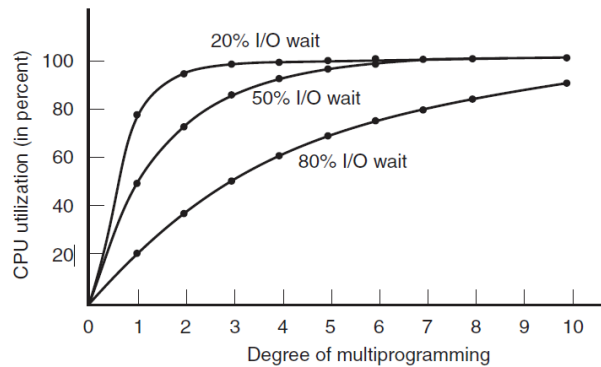


Figure 1: Caption text describing the image

Key Trend

As n increases, p^n becomes smaller, so CPU utilization increases.

Situation	Result
Small n	Higher chance that all processes are waiting, so CPU may be idle more often
Large n	Lower chance that all processes are waiting, so CPU utilization improves
Very high I/O wait (p near 1)	Many processes are needed to keep CPU waste low

4. Why High I/O Wait Is Common

Real Systems

Processes often spend a large fraction of time in I/O wait. This is common not only for disk-heavy server programs, but also for interactive processes waiting for a user to type or click something.

Example

If a process is waiting for user input at a terminal, that time counts as I/O wait. So values such as $p = 0.8$ are not unusual.

5. Worked Example: CPU Utilization

Example

Suppose each process waits for I/O 80% of the time, so:

$$p = 0.8$$

Then:

$$\text{CPU utilization} = 1 - 0.8^n$$

n	Idle Probability p^n	CPU Utilization $1 - p^n$
3	$0.8^3 = 0.512$	$48.8\% \approx 49\%$
7	$0.8^7 \approx 0.210$	79.0%
11	$0.8^{11} \approx 0.086$	91.4%

Interpretation

This shows that adding more processes improves utilization, but the improvement becomes smaller after a while.

6. Memory Expansion Example

Given System

Suppose a computer has:

- 8 GB total memory,
- 2 GB used by the operating system,
- each user program requiring 2 GB.

That leaves room for:

$$\frac{8 - 2}{2} = 3$$

user processes in memory.

Initial Utilization

With $p = 0.8$ and $n = 3$:

$$\text{CPU utilization} = 1 - 0.8^3 \approx 49\%$$

After Adding 8 GB

If another 8 GB of memory is added, total memory becomes 16 GB. Then:

$$\frac{16 - 2}{2} = 7$$

user processes can be kept in memory.

Now:

$$\text{CPU utilization} = 1 - 0.8^7 \approx 79\%$$

Performance Gain

CPU utilization rises from about 49% to 79%, so throughput improves by about 30%.

Adding Another 8 GB

If memory increases again from 16 GB to 24 GB, then:

$$\frac{24 - 2}{2} = 11$$

processes can fit in memory.

Utilization becomes:

$$\text{CPU utilization} = 1 - 0.8^{11} \approx 91\%$$

Diminishing Returns

The second memory upgrade raises utilization only from about 79% to 91%, an improvement of about 12%. This shows **diminishing returns**: the first upgrade helps more than the second.

7. Limits of the Model

Approximation

This probabilistic model is only an approximation. It assumes that all processes are independent.

Why It Is Not Exact

With a single CPU, several processes cannot all run at once. If one process becomes ready while another is already using the CPU, it must wait. So the processes are not completely independent.

More Accurate Approach

A more precise model can be built using **queueing theory**. Even so, the simple model still captures the key idea: multiprogramming reduces CPU idle time.

8. Main Conclusion

Takeaway

Multiprogramming improves performance by letting one process use the CPU while others wait for I/O. The formula $1 - p^n$ gives a simple way to estimate CPU utilization and reason about tradeoffs such as whether adding more memory is worth it.

9. Quick Review

Exam-Focused Points

- Multiprogramming improves CPU utilization by reducing idle time.
- If a process waits for I/O a fraction p of the time, then the probability all n processes are waiting is p^n .
- CPU utilization is approximated by $1 - p^n$.
- n is the degree of multiprogramming.
- High I/O wait values such as 80% are common in interactive and I/O-heavy systems.
- Increasing the number of processes in memory improves utilization.
- The improvement shows diminishing returns as n becomes large.
- The model is approximate because processes are not fully independent on a single CPU.